

Algoritmos de Ordenamiento y Archivos de Texto

Semana 6 — Ordenamiento por selección, inserción y burbuja ($O(n^2)$), merge sort ($O(n \log n)$), archivos de texto, compiladores e intérpretes, y ciclo de vida de procesos

1. El Problema del Ordenamiento

El **ordenamiento** es, junto con la búsqueda, uno de los problemas más estudiados en la historia de la computación. Se estima que más del 25% del tiempo de cómputo en aplicaciones comerciales se dedica a ordenar datos. La razón es directa: muchas operaciones posteriores —búsqueda binaria, eliminación de duplicados, generación de informes, fusión de archivos— requieren que los datos estén ordenados como precondition.

Formalmente, el problema se define así: dada una secuencia de n elementos con una relación de orden total (por ejemplo, $\leq$ para números), producir una permutación de la secuencia tal que los elementos queden dispuestos en orden no decreciente.

Contrato de la función de ordenamiento

Entrada: una secuencia `datos` de n elementos comparables entre sí.

Salida: la misma secuencia reordenada tal que `datos[0]  $\leq$  datos[1]  $\leq$  ...  $\leq$  datos[n-1]`.

Precondición: los elementos deben admitir comparación con $<$ y $\leq$. Para ordenamiento *in-place*, la secuencia debe ser mutable (lista, no tupla).

Poscondición: la secuencia resultante es una permutación de la original (no se pierden ni se inventan elementos) y está ordenada.

¿Por qué ordenar? Conexión con la búsqueda binaria

En la Semana 5 estudiamos que la **búsqueda binaria** requiere datos ordenados como precondition, pero es dramáticamente más eficiente que la búsqueda lineal: $O(\log n)$ frente a $O(n)$. Esta semana estudiamos el *costo* de cumplir esa precondition: ¿cuánto cuesta ordenar? Si ordenar cuesta $O(n^2)$ y buscar $O(\log n)$, conviene ordenar una vez y buscar muchas veces. Si ordenar cuesta $O(n \log n)$, la inversión se amortiza aún más rápido. La pregunta clave es: **¿cuántas búsquedas vamos a hacer sobre los mismos datos?**

1.1. Clasificación de algoritmos de ordenamiento

Los algoritmos de ordenamiento se clasifican según varios criterios que determinan su idoneidad para distintos contextos:

- **Comparativos vs. no comparativos:** los comparativos determinan el orden mediante comparaciones entre pares de elementos (los que estudiamos aquí); los no comparativos (counting sort, radix sort) explotan propiedades especiales de los datos.
- **In-place vs. con memoria auxiliar:** un algoritmo es *in-place* si solo usa una cantidad constante de memoria adicional $O(1)$; caso contrario requiere memoria proporcional a los datos (por ejemplo, merge sort usa $O(n)$ adicional).
- **Estables vs. inestables:** un algoritmo es **estable** si preserva el orden relativo de elementos con claves iguales. Esto importa cuando se ordena por múltiples criterios sucesivos.

Estabilidad: ¿por qué importa?

Supongamos que tenemos una lista de alumnos ya ordenada alfabéticamente y queremos reordenarla por nota. Si el algoritmo es **estable**, los alumnos con la misma nota quedarán en orden alfabético entre sí (el orden previo se preserva). Si es **inestable**, el orden entre alumnos con igual nota es impredecible.

2. Ordenamiento por Selección (*Selection Sort*)

2.1. Idea intuitiva

El algoritmo de **selección** imita lo que haría una persona al ordenar un mazo de cartas: en cada pasada, recorre las cartas desordenadas, **selecciona la menor** y la coloca en su posición definitiva. Después de k pasadas, las primeras k posiciones contienen los k menores elementos en orden correcto.



2.2. Pseudocódigo

Algoritmo ordenamiento_seleccion(datos, n)

Para $i \leftarrow 0$ **hasta** $n - 2$:

$pos_min \leftarrow i$

Para $j \leftarrow i + 1$ **hasta** $n - 1$:

Si $datos[j] < datos[pos_min]$:

$pos_min \leftarrow j$

Si $pos_min \neq i$:

 intercambiar $datos[i] \leftrightarrow datos[pos_min]$

Ambos bucles son «Para» (for/range): la cantidad de iteraciones está

determinada antes de entrar.

2.3. Implementación en Python

```
def ordenamiento_seleccion(datos):  
    '''Ordena la lista datos in-place usando selección directa.  
  
    Precondición: datos es una lista de elementos comparables.  
    Poscondición: datos queda ordenada en forma no decreciente.  
    Complejidad:  $O(n^2)$  comparaciones en todos los casos.  
    '''  
    n = len(datos)  
    for i in range(n - 1):  
        pos_min = i  
        for j in range(i + 1, n):  
            if datos[j] < datos[pos_min]:  
                pos_min = j  
  
        if pos_min != i:  
            datos[i], datos[pos_min] = datos[pos_min], datos[i]
```

for/range() VS. while: criterio de elección

En la Semana 3 establecimos la norma: `for` con `range()` para recorridos **determinados** (se conoce de antemano cuántas iteraciones se harán) y `while` para recorridos **indeterminados** (la condición de parada depende de los datos o de múltiples criterios simultáneos). En los algoritmos de ordenamiento aplicamos esta norma de forma consistente:

- **Selección** — ambos bucles son `for/range()`: el bucle externo recorre exactamente $n-1$ pasadas y el interno recorre desde $i+1$ hasta $n-1$.
- **Inserción** — bucle externo `for/range(1, n)` (exactamente $n-1$ elementos a insertar); bucle interno `while` (condición compuesta: $j \geq 0$ and `datos[j] > clave`).
- **Burbuja** — bucle externo `while` (dos condiciones: pasadas restantes y bandera de intercambio); bucle interno `for/range(n-1-i)` (cantidad de

comparaciones determinada para cada pasada).

2.4. Traza manual

Datos iniciales: [64, 25, 12, 22, 11]

Pasada (i)	Sublista a explorar	pos_min	Intercambio	Estado de datos
0	[64, 25, 12, 22, 11]	4 (valor 11)	datos[0] ↔ datos[4]	[11 , 25, 12, 22, 64]
1	[25, 12, 22, 64]	2 (valor 12)	datos[1] ↔ datos[2]	[11, 12 , 25, 22, 64]
2	[25, 22, 64]	3 (valor 22)	datos[2] ↔ datos[3]	[11, 12, 22 , 25, 64]
3	[25, 64]	3 (valor 25)	ninguno (ya en posición)	[11, 12, 22, 25, 64]

2.5. Invariante de bucle y correctitud

Invariante del bucle externo

Al comenzar la iteración i del bucle externo, las primeras i posiciones de datos contienen los i menores elementos de la secuencia original, en orden no decreciente, cada uno en su posición definitiva.

Inicialización: cuando $i = 0$, la zona ordenada está vacía — el invariante se cumple trivialmente.

Mantenimiento: el bucle interno encuentra el mínimo de $\text{datos}[i..n-1]$ y lo ubica en $\text{datos}[i]$; así la zona ordenada crece en un elemento y el invariante se mantiene.

Terminación: cuando $i = n - 1$, las primeras $n - 1$ posiciones están correctas, y el último elemento es necesariamente el mayor — la secuencia completa está ordenada.

2.6. Complejidad

En la pasada i , el bucle interno realiza $n - i - 1$ comparaciones. El total es:

$$(n-1) + (n-2) + \dots + 1 + 0 = \mathbf{n(n-1)/2} \text{ comparaciones} \approx \mathbf{O(n^2)}$$

Observación importante: la cantidad de comparaciones es **siempre la misma**, independientemente de si los datos ya estaban ordenados, desordenados o en orden inverso. La selección no se adapta al estado inicial de los datos.

¿Es estable el ordenamiento por selección?

No. Consideremos $[3a, 3b, 1]$ (donde $3a$ y $3b$ son iguales pero distinguimos su orden original). En la pasada 0, el mínimo es 1 $\rightarrow$ swap con $3a \rightarrow [1, 3b, 3a]$. El orden relativo de los dos 3 se invirtió.

3. Ordenamiento por Inserción (*Insertion Sort*)

3.1. Idea intuitiva

El algoritmo de **inserción** imita cómo se ordenan cartas en la mano: se toma cada carta nueva y se la **inserta en la posición correcta** dentro de las cartas ya ordenadas, desplazando las mayores hacia la derecha para hacer lugar.

Ordenamiento por Inserción: paso a paso



3.2. Implementación en Python

```
def ordenamiento_insercion(datos):  
    '''Ordena la lista datos in-place usando inserción directa.  
  
    Precondición: datos es una lista de elementos comparables.  
    Poscondición: datos queda ordenada en forma no decreciente.  
    Complejidad:  $O(n^2)$  peor caso,  $O(n)$  mejor caso (ya  
ordenada).  
    Estabilidad: estable.  
    '''  
    n = len(datos)  
    for i in range(1, n):  
        clave = datos[i]  
        j = i - 1  
        while j >= 0 and datos[j] > clave:  
            datos[j + 1] = datos[j] # desplazar a la derecha  
            j -= 1  
        datos[j + 1] = clave # insertar en el hueco
```

¿Por qué la condición del bucle interno es compuesta?

La condición $j \geq 0$ and $\text{datos}[j] > \text{clave}$ combina dos motivos de parada: llegar al inicio de la lista ($j < 0$) o encontrar un elemento menor o igual a la clave ($\text{datos}[j] \leq \text{clave}$). Al salir del bucle, la inspección de cuál condición se falsificó no es necesaria porque en ambos casos $j + 1$ es la posición correcta para insertar la clave. La evaluación en cortocircuito garantiza que $\text{datos}[j]$ no se evalúe cuando $j < 0$.

3.3. Complejidad

- **Peor caso** (datos en orden inverso): cada elemento se compara con todos los anteriores $\Rightarrow n(n-1)/2$ comparaciones $\Rightarrow O(n^2)$.
- **Mejor caso** (datos ya ordenados): el bucle interno nunca ejecuta su cuerpo $\Rightarrow$ solo $n-1$ comparaciones $\Rightarrow O(n)$.
- **Caso promedio**: cada elemento se inserta en promedio a la mitad de la zona ordenada $\Rightarrow n(n-1)/4$ comparaciones $\Rightarrow O(n^2)$.

Adaptabilidad: la gran ventaja de la inserción

A diferencia de selección, que siempre realiza $O(n^2)$ comparaciones, la inserción se **adapta** al grado de desorden de los datos. Si la secuencia está «casi ordenada» (pocos elementos fuera de lugar), inserción es casi lineal. Esto la hace excelente para **listas pequeñas** o **listas casi ordenadas**, razón por la cual el algoritmo `sorted()` de Python (Timsort) la usa internamente para sublistas cortas.

4. Ordenamiento Burbuja (*Bubble Sort*)

4.1. Idea intuitiva

El algoritmo **burbuja** recorre la secuencia repetidamente, comparando pares de elementos adyacentes e intercambiándolos si están en orden incorrecto. En cada

pasada, el mayor elemento «burbujea» hacia el final, como una burbuja que sube a la superficie. Después de k pasadas, los últimos k elementos están en su posición definitiva.



4.2. Implementación con optimización de parada temprana

```
def ordenamiento_burbuja(datos):  
    '''Ordena la lista datos in-place usando burbuja con parada  
    temprana.
```

Precondición: datos es una lista de elementos comparables.

Poscondición: datos queda ordenada en forma no decreciente.

Complejidad: $O(n^2)$ peor caso, $O(n)$ mejor caso (con optimización).

Estabilidad: estable.

'''

```
n = len(datos)
```

```
i = 0
```

```
hubo_intercambio = True
```

```
while i < n - 1 and hubo_intercambio:
```

```
    hubo_intercambio = False
```

```
    for j in range(n - 1 - i):
```

```
        if datos[j] > datos[j + 1]:
```

```

        datos[j], datos[j + 1] = datos[j + 1], datos[j]
        hubo_intercambio = True

    i += 1

# Inspección post-bucle:
# Si salió por hubo_intercambio == False: la pasada no hizo
swaps → ya está ordenado
# Si salió por i == n-1: se completaron todas las pasadas →
ordenado por correctitud

```

Sobre la condición compuesta del bucle externo

El bucle externo tiene dos condiciones: $i < n - 1$ (no se completaron todas las pasadas) **y** `hubo_intercambio` (la pasada anterior realizó al menos un swap). La bandera `hubo_intercambio` permite la **parada temprana**: si una pasada completa no realiza ningún intercambio, la secuencia ya está ordenada y no tiene sentido continuar. Esto convierte al mejor caso (datos ya ordenados) en $O(n)$ —una sola pasada sin intercambios—. Nótese que **no usamos break**: la condición compuesta del `while` maneja ambos criterios de terminación.

4.3. Complejidad de burbuja

- **Peor caso** (orden inverso): $n-1$ pasadas, cada una con comparaciones decrecientes $\Rightarrow n(n-1)/2 \Rightarrow O(n^2)$.
- **Mejor caso** (ya ordenado, con optimización): una sola pasada sin intercambios $\Rightarrow n-1$ comparaciones $\Rightarrow O(n)$.
- **Sin optimización**: siempre $O(n^2)$, sin importar el estado inicial.

5. Comparación de los Algoritmos Cuadráticos

Aspecto	Selección	Inserción	Burbuja
Estrategia	Buscar el mínimo,	Tomar el siguiente,	Comparar adyacentes,

Aspecto	Selección	Inserción	Burbuja
	colocarlo	insertarlo en su lugar	intercambiar si desordenados
Peor caso	$O(n^2)$	$O(n^2)$	$O(n^2)$
Mejor caso	$O(n^2)$ — no se adapta	$O(n)$ — ya ordenada	$O(n)$ — con optimización
Intercambios (swaps)	$O(n)$ — uno por pasada como máximo	$O(n^2)$ — desplazamientos (no swaps)	$O(n^2)$ — muchos swaps
Estabilidad	No estable	Estable	Estable
Memoria extra	$O(1)$ — in-place	$O(1)$ — in-place	$O(1)$ — in-place
¿Cuándo conviene?	Cuando los intercambios son costosos (el swap es caro)	Listas pequeñas o casi ordenadas; excelente rendimiento adaptativo	Uso pedagógico; en la práctica es superado por inserción

La barrera $O(n^2)$: ¿se puede hacer mejor?

Los tres algoritmos anteriores son $O(n^2)$ en el peor caso. ¿Es inevitable? No. Se puede demostrar que cualquier algoritmo de ordenamiento basado en comparaciones necesita al menos $O(n \log n)$ comparaciones en el peor caso. Esto significa que $O(n \log n)$ es un **límite inferior teórico** para los algoritmos comparativos —y hay algoritmos que lo alcanzan. El primero que estudiaremos es **merge sort**.

6. Merge Sort: Divide y Vencerás

6.1. La estrategia Divide y Vencerás

El paradigma **divide y vencerás** (*divide and conquer*) es una estrategia general de diseño de algoritmos que consta de tres pasos:

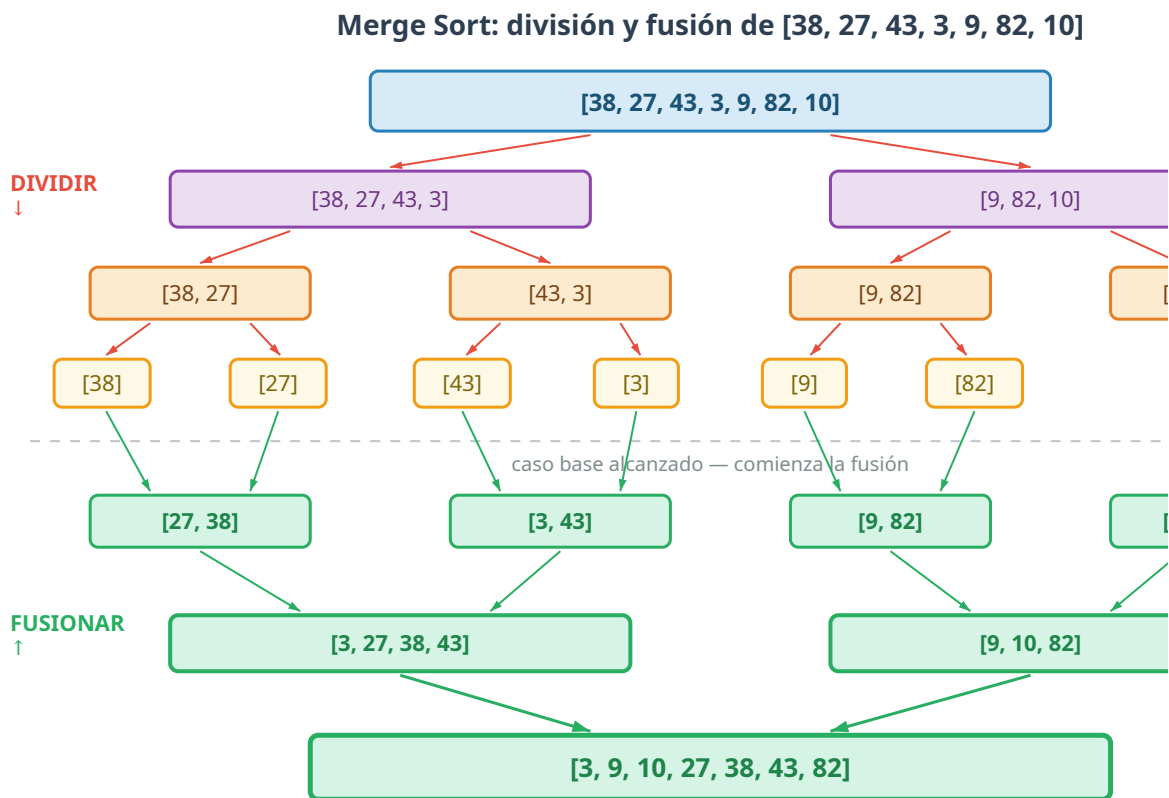
1. **Dividir:** descomponer el problema en subproblemas más pequeños del mismo tipo.
2. **Vencer:** resolver cada subproblema recursivamente (o directamente si es suficientemente pequeño).
3. **Combinar:** fusionar las soluciones de los subproblemas para obtener la solución del problema original.

¿Por qué funciona? La idea de la recursividad

Merge sort es un algoritmo **recursivo**. La recursividad se estudia en profundidad en la Semana 8 (caso base, caso recursivo, árbol de llamadas, correctitud por inducción). Aquí la introducimos aplicada a un problema concreto: si entendemos cómo funciona merge sort, tendremos una base sólida para comprender la recursividad como paradigma general.

6.2. Merge Sort paso a paso

La idea de merge sort es elegantemente simple: dividir la secuencia por la mitad, ordenar recursivamente cada mitad, y luego **fusionar** (*merge*) las dos mitades ordenadas en una sola secuencia ordenada. El caso base es una secuencia de 0 o 1 elementos, que ya está trivialmente ordenada.



6.3. La operación clave: fusión (*merge*)

La operación de **fusión** toma dos secuencias *ya ordenadas* y produce una única secuencia ordenada. La idea es simple: comparar los primeros elementos de ambas secuencias, copiar el menor al resultado, y avanzar en la secuencia de donde se copió. Cuando una se agota, copiar el resto de la otra.

```
def fusionar(izq, der):
    '''Fusiona dos listas ordenadas en una sola lista ordenada.

    Precondición: izq y der están ordenadas en forma no
    decreciente.
    Poscondición: retorna una nueva lista con todos los
    elementos
                    de izq y der, ordenados. No modifica las
    originales.
    Complejidad: O(n) donde n = len(izq) + len(der).
    '''
    resultado = []
```

```

i = 0
j = 0

# Fase 1: comparar y copiar el menor de ambas listas
while i < len(izq) and j < len(der):
    if izq[i] <= der[j]:      # <= para estabilidad
        resultado.append(izq[i])
        i += 1
    else:
        resultado.append(der[j])
        j += 1

# Fase 2: copiar los restantes de la lista que no se agotó
while i < len(izq):
    resultado.append(izq[i])
    i += 1
while j < len(der):
    resultado.append(der[j])
    j += 1

return resultado

```

¿Por qué <= y no < en la comparación?

Usar <= (y no <) garantiza la **estabilidad** del algoritmo: cuando ambos elementos son iguales, se copia primero el de la sublista izquierda, preservando el orden relativo original.

6.4. Implementación completa de merge sort

```

def merge_sort(datos):
    '''Ordena la lista datos usando el algoritmo merge sort.

```

Precondición: datos es una lista de elementos comparables.

Poscondición: retorna una NUEVA lista con los mismos elementos, ordenados.

```

Complejidad:  $O(n \log n)$  en todos los casos.
Memoria auxiliar:  $O(n)$ .
Estabilidad: estable.
'''

n = len(datos)

# Caso base: lista de 0 o 1 elementos ya está ordenada
if n <= 1:
    return datos[:] # retornar una copia (no modificar la
original)

# Dividir: partir por la mitad
medio = n >> 1 # equivalente a  $n // 2$ , pero más
eficiente
izq = merge_sort(datos[:medio])
der = merge_sort(datos[medio:])

# Combinar: fusionar las dos mitades ordenadas
return fusionar(izq, der)

```

Nota sobre $n \gg 1$

La expresión $n \gg 1$ (desplazamiento a la derecha de 1 bit) es equivalente a $n // 2$ para enteros no negativos. A nivel de la ALU, un desplazamiento de bits es una sola instrucción del procesador, mientras que la división entera puede requerir múltiples ciclos. En la Semana 1 vimos que la ALU realiza desplazamientos como operación primitiva; aquí aplicamos ese conocimiento. Si bien en Python la diferencia de rendimiento es mínima (la PVM interpreta ambas operaciones), el hábito de pensar en eficiencia a nivel de máquina es una competencia valiosa para un ingeniero.

6.5. Complejidad de merge sort

¿Por qué merge sort es $O(n \log n)$? Analicemos la estructura recursiva:

- **Nivel 0:** 1 fusión de n elementos $\Rightarrow O(n)$ operaciones.
- **Nivel 1:** 2 fusiones de $n/2$ elementos cada una $\Rightarrow O(n)$ operaciones en total.

- **Nivel 2:** 4 fusiones de $n/4$ elementos cada una $\Rightarrow O(n)$ operaciones en total.
- **En general:** en cada nivel se fusionan en total n elementos $\Rightarrow O(n)$ por nivel.
- **¿Cuántos niveles hay?** En cada nivel se divide por 2, empezando en n y llegando a 1 $\Rightarrow$ hay $\log_2(n)$ niveles.

$$\text{Total} = O(n) \times O(\log n) = O(n \log n)$$

Esta complejidad es la misma en el mejor, peor y caso promedio: merge sort siempre divide y siempre fusiona, sin importar el orden inicial de los datos.

Costo de la eficiencia: memoria auxiliar $O(n)$

Merge sort necesita crear listas auxiliares durante la fusión. En total, la memoria adicional es del orden $O(n)$. Este es el **trade-off**: se gana eficiencia temporal ($O(n \log n)$ en lugar de $O(n^2)$) pero se paga con espacio adicional. Los algoritmos cuadráticos, en cambio, son in-place ($O(1)$ memoria extra).

7. Quick Sort: Particionar y Vencer

Quick sort es otro algoritmo $O(n \log n)$ basado en divide y vencerás, pero con una estrategia radicalmente diferente a la de merge sort. En lugar de dividir por la mitad y luego *fusionar*, quick sort elige un **pivote**, **particiona** la secuencia colocando todos los menores a la izquierda y todos los mayores a la derecha, y luego ordena recursivamente cada partición. El pivote queda en su posición definitiva tras la partición —sin necesidad de una operación de fusión posterior.

Los tres pasos de quick sort

1. **Elegir un pivote:** seleccionar un elemento de la secuencia (el último, el primero, uno aleatorio, o la mediana de tres).
2. **Particionar:** reorganizar los elementos de modo que todos los menores al pivote queden a su izquierda y todos los mayores a su derecha. El pivote queda en su **posición definitiva**.
3. **Recurrir:** aplicar quick sort a la sublista izquierda y a la sublista derecha (sin incluir el pivote, que ya está bien ubicado).

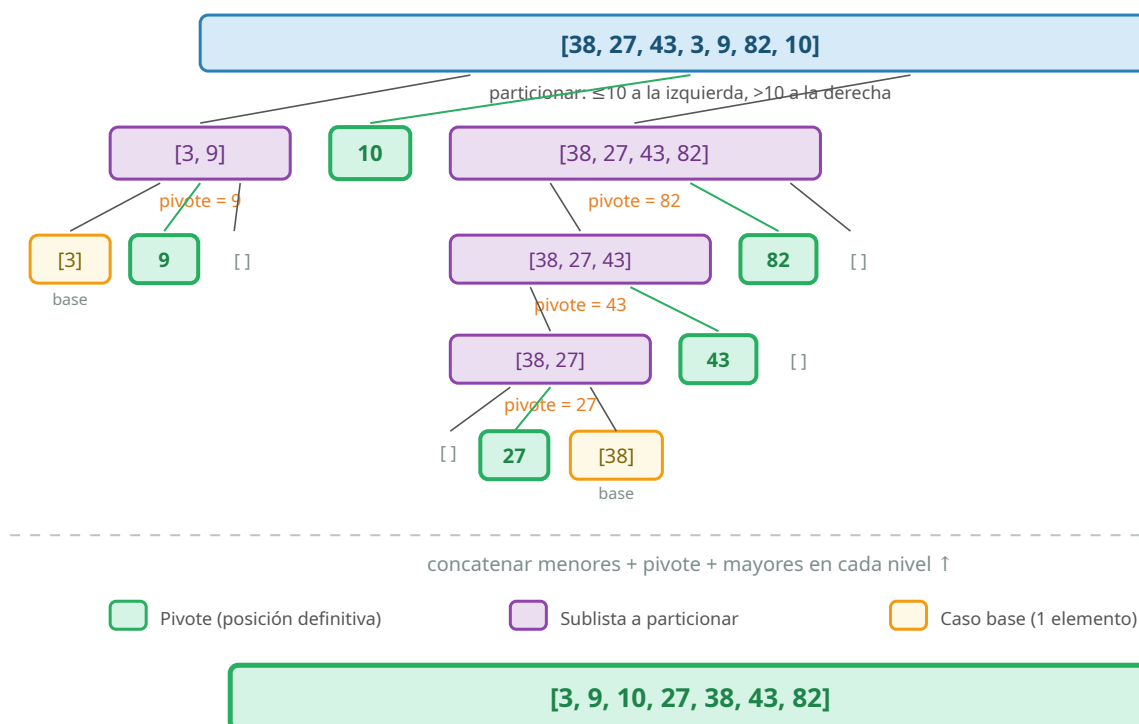
Merge sort vs. quick sort: ¿dónde está el trabajo?

En merge sort, **dividir es trivial** (partir por la mitad) y **combinar es costoso** (fusionar dos listas ordenadas). En quick sort es al revés: **dividir es costoso** (particionar comparando cada elemento con el pivote) y **combinar es trivial** (no hay que hacer nada —la concatenación de «menores + pivote + mayores» ya está ordenada).

7.1. Ejemplo con buen pivote

Usemos la misma lista que en merge sort: [38, 27, 43, 3, 9, 82, 10]. Si en cada nivel elegimos como pivote el **último elemento** de la sublista:

Quick Sort (pivote = último elemento): [38, 27, 43, 3, 9, 82, 10]



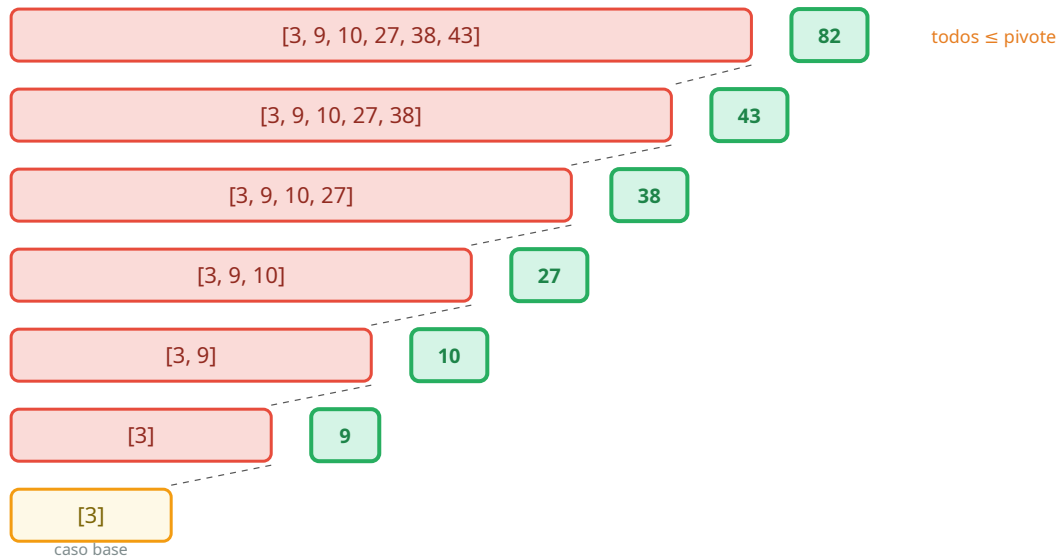
Observación clave

Cada pivote queda en su posición definitiva inmediatamente después de la partición (marcados en verde). No se necesita una fusión posterior: la concatenación «menores + pivote + mayores» es la solución. En este ejemplo, con pivote = último elemento, las particiones quedaron razonablemente equilibradas y la recursión tuvo poca profundidad.

7.2. Ejemplo con mal pivote: el caso degenerado

¿Qué sucede si la elección de pivote es consistentemente mala? Supongamos que la misma lista ya estuviese ordenada: [3, 9, 10, 27, 38, 43, 82] y usamos pivote = último. En cada paso, el pivote sería el mayor, la partición izquierda tendría $n-1$ elementos y la derecha estaría vacía:

Quick Sort degenerado: lista ya ordenada, pivote = último



$n - 1$ niveles de recursión $\times O(n)$ comparaciones por nivel = $O(n^2)$

El peor caso de quick sort: $O(n^2)$

Cuando el pivote es siempre el mayor (o el menor) elemento, la partición deja $n-1$ elementos de un lado y 0 del otro. La recursión tiene profundidad $n-1$ (en lugar de $\log n$), y en cada nivel se hacen $O(n)$ comparaciones, dando un total de $O(n^2)$. Esto ocurre, irónicamente, cuando los datos *ya están ordenados* y se elige el último elemento como pivote. Las estrategias para mitigar este riesgo incluyen: elegir el pivote **aleatoriamente**, usar la **mediana de tres** (primer, último y medio elemento), o cambiar a inserción para sublistas pequeñas.

7.3. Pseudocódigo e implementación simplificada

Algoritmo quick_sort(datos)

Si longitud(datos) ≤ 1:

Retornar datos

pivote ← datos[longitud(datos) - 1]

menores ← [x para cada x en datos[0..n-2] si x ≤ pivote]

mayores ← [x para cada x en datos[0..n-2] si x > pivote]

```
Retornar quick_sort(menores) + [pivote] + quick_sort(mayores)
```

```
def quick_sort(datos):  
    '''Ordena la lista datos usando quick sort (versión  
simplificada).  
  
    Precondición: datos es una lista de elementos comparables.  
    Poscondición: retorna una NUEVA lista con los mismos  
elementos, ordenados.  
    Complejidad:  $O(n \log n)$  caso promedio,  $O(n^2)$  peor caso.  
    Nota: esta versión NO es in-place (crea listas auxiliares).  
        La versión in-place clásica se estudiará en la Semana  
11.  
    '''  
    n = len(datos)  
    if n <= 1:  
        return datos[:]  
  
    pivote = datos[n - 1]  
    menores = []  
    mayores = []  
    for i in range(n - 1):  
        if datos[i] <= pivote:  
            menores.append(datos[i])  
        else:  
            mayores.append(datos[i])  
  
    return quick_sort(menores) + [pivote] + quick_sort(mayores)
```

Versión simplificada vs. versión in-place

La implementación anterior es conceptualmente clara pero **no es in-place**: crea listas `menores` y `mayores` en cada llamada recursiva, con un costo de memoria $O(n)$ similar al de merge sort. La ventaja práctica de quick sort (memoria $O(\log n)$ usando solo la pila de recursión) se logra con la versión *in-place* clásica, que particiona intercambiando elementos dentro de la misma lista usando dos punteros. Esta versión se estudia en profundidad en la

7.4. Comparación con merge sort

Aspecto	Merge Sort	Quick Sort
Dónde está el trabajo	En la <i>fusión</i> (después de recurrir)	En la <i>partición</i> (antes de recurrir)
Peor caso	$O(n \log n)$ — siempre	$O(n^2)$ — pivote malo
Caso promedio	$O(n \log n)$	$O(n \log n)$
Memoria auxiliar	$O(n)$ — no in-place	$O(\log n)$ — solo la pila (versión in-place)
Estabilidad	Estable	No estable (en su versión in-place clásica)
Localidad de caché	Menor (listas auxiliares dispersas)	Mayor (opera sobre datos contiguos)
En la práctica	Predecible; ideal para datos en disco	Usualmente más rápido en RAM; base de <code>qsort()</code> en C

8. Tabla Resumen de Complejidades

Algoritmo	Mejor caso	Caso promedio	Peor caso	Memoria	Estable
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí

Algoritmo	Mejor caso	Caso promedio	Peor caso	Memoria	Estable
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Python sorted()	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí

Timsort: lo mejor de ambos mundos

La función `sorted()` y el método `.sort()` de Python usan **Timsort**, un algoritmo híbrido diseñado por Tim Peters en 2002. Timsort combina inserción (para sublistas cortas) y merge sort (para la fusión global), aprovechando las «corridas» (*runs*) de elementos ya parcialmente ordenados que suelen existir en datos reales. Su mejor caso es $O(n)$ cuando los datos ya están ordenados, y su peor caso es $O(n \log n)$. Es estable.

9. Archivos de Texto en Python

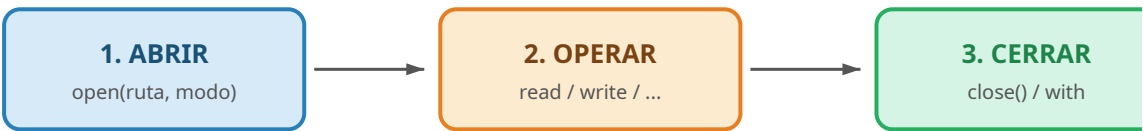
Hasta ahora, nuestros programas reciben datos del teclado y muestran resultados en pantalla. Esa información es **volátil**: desaparece cuando el programa termina. Los **archivos** permiten que los datos **persistan** más allá de la ejecución del programa, almacenándose en medios no volátiles (disco duro, SSD).

Conexión holística: memoria volátil y no volátil

En la Semana 5 estudiamos la **jerarquía de memoria**: registros, caché, RAM (volátil) y disco (no volátil). Los archivos residen en el nivel más bajo de la jerarquía —son lentos de acceder pero persisten sin alimentación eléctrica. El **sistema operativo** gestiona todo el acceso a archivos mediante el *sistema de ficheros* (file system), que traduce las operaciones lógicas (abrir, leer, escribir) en operaciones físicas sobre el disco.

9.1. Ciclo de vida de un archivo

El trabajo con archivos sigue un patrón estricto de tres fases:



El SO asigna un descriptor de archivo (file descriptor) al abrir y lo libera al cerrar.

9.2. Modos de apertura

Modo	Descripción	Si el archivo no existe	Si el archivo existe
'r'	Lectura (por defecto)	Error FileNotFoundError	Lee desde el inicio
'w'	Escritura	Lo crea	Borra el contenido previo
'a'	Añadir (append)	Lo crea	Escribe al final
'r+'	Lectura y escritura	Error	Lee/escribe desde el inicio

Peligro con el modo 'w'

El modo 'w' **destruye el contenido previo** del archivo al abrirlo. Si se necesita añadir datos sin perder los existentes, usar 'a'. Este es un error frecuente y difícil de revertir: una vez borrado, el contenido no se puede recuperar (el SO marca los bloques del disco como libres).

9.3. Context managers: la sentencia with

La forma recomendada de trabajar con archivos en Python es mediante un **context manager** (*gestor de contexto*), que garantiza que el archivo se cierre correctamente incluso si ocurre un error durante la operación:

```
# Forma recomendada: with garantiza el cierre
with open('datos.txt', 'r') as archivo:
    contenido = archivo.read()
    # Al salir del bloque with, archivo.close() se ejecuta
    automáticamente

# Forma manual (NO recomendada): si ocurre un error, el archivo
# queda abierto
archivo = open('datos.txt', 'r')
contenido = archivo.read()
archivo.close() # si read() falla, esta línea no se ejecuta
```

¿Qué hace with internamente?

La sentencia with invoca el método `__enter__()` del objeto archivo al entrar en el bloque y `__exit__()` al salir, garantizando que `close()` se llame siempre. A nivel del SO, cerrar un archivo libera el descriptor de archivo y vacía (*flush*) los datos que pudieran estar en el búfer del SO hacia el disco.

9.4. Lectura de archivos de texto

Python ofrece varias formas de leer un archivo de texto:

```
# Método 1: leer todo el contenido como una cadena
with open('datos.txt', 'r') as archivo:
    contenido = archivo.read()      # un solo string con todo
    el texto
```

```
# Método 2: leer todas las líneas como lista de strings
with open('datos.txt', 'r') as archivo:
    lineas = archivo.readlines()    # ['línea1\n', 'línea2\n',
    ...]
```

Método 3: iterar línea a línea (RECOMENDADO para archivos grandes)

```
with open('datos.txt', 'r') as archivo:
    for linea in archivo:
        linea = linea.strip()        # eliminar '\n' final
        # procesar la línea
```

¿Por qué iterar línea a línea es preferible?

Los métodos `read()` y `readlines()` cargan **todo el archivo en memoria RAM**. Si el archivo tiene 10 GB, se necesitan 10 GB de RAM. En cambio, iterar con `for linea in archivo` lee una línea a la vez, usando memoria constante independientemente del tamaño del archivo. Esta diferencia es crucial en aplicaciones reales.

9.5. Escritura de archivos de texto

```
# Escribir líneas una a una
with open('resultados.txt', 'w') as archivo:
    archivo.write('Línea 1\n')
    archivo.write('Línea 2\n')
    archivo.write(f'El resultado es: {resultado}\n')
```

9.6. Ejemplo integrador: leer, ordenar y escribir

Veamos un programa completo que lee números de un archivo, los ordena con merge sort y escribe el resultado en otro archivo. Esto conecta los dos temas principales de la semana: ordenamiento y archivos.

```
def leer_numeros(ruta):
    '''Lee un archivo de texto con un número por línea y
    retorna una lista de enteros.

    Precondición: el archivo existe y cada línea contiene un
    entero válido.
    Poscondición: retorna la lista de enteros leídos, en el
    orden del archivo.
    '''
    numeros = []
    with open(ruta, 'r') as archivo:
        for linea in archivo:
            numeros.append(int(linea.strip()))
    return numeros

def escribir_numeros(ruta, numeros):
    '''Escribe una lista de enteros en un archivo, uno por
    línea.

    Precondición: ruta es una ruta válida de archivo.
    Poscondición: el archivo contiene los enteros de la lista,
    uno por línea.
    '''
    with open(ruta, 'w') as archivo:
        for i in range(len(numeros)):
            archivo.write(f'{numeros[i]}\n')

# — Sección algorítmica —————
# Prólogo: lectura de datos
datos = leer_numeros('entrada.txt')
print(f'Leídos {len(datos)} números.')

# Resolución: ordenar
```

```
datos_ordenados = merge_sort(datos)

# Epílogo: escritura de resultados
escribir_numeros('salida.txt', datos_ordenados)
print('Archivo ordenado generado: salida.txt')
```

Patrón: Prólogo → Resolución → Epílogo con archivos

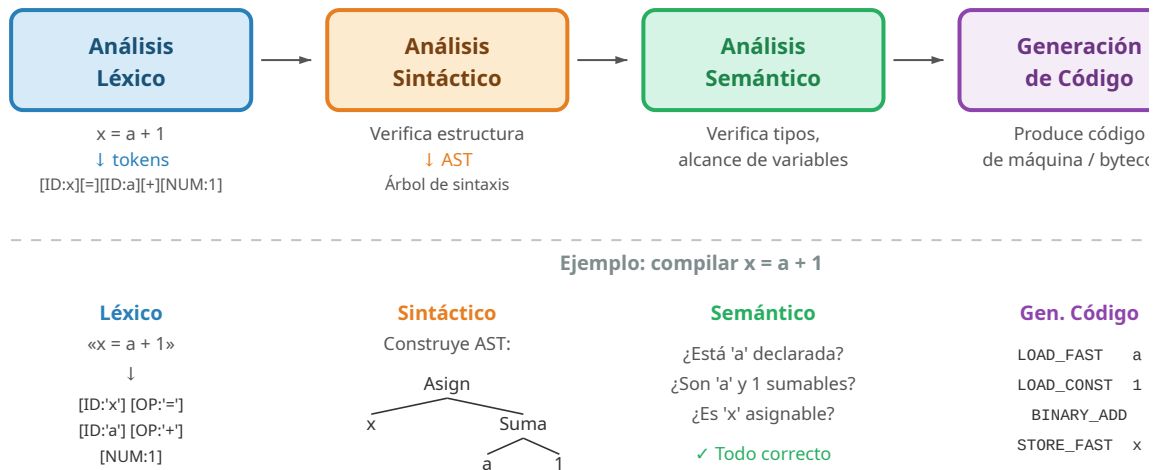
Nuestro modelo de programa se extiende naturalmente al trabajo con archivos: el **prólogo** abre y lee los datos de entrada, la **resolución** aplica el algoritmo, y el **epílogo** escribe los resultados y presenta la salida. Las funciones `leer_numeros` y `escribir_numeros` encapsulan las operaciones de E/S, separando la lógica de acceso a archivos de la lógica algorítmica (principio de *separación de responsabilidades*).

10. [Holístico] Compiladores e Intérpretes

En la Semana 4 introdujimos la distinción entre compiladores e intérpretes y vimos cómo Python es un caso *híbrido*. Esta semana profundizamos en las **fases de compilación**, conectando este conocimiento con nuestra comprensión de cómo el código fuente se transforma hasta llegar a la CPU.

10.1. Las fases del proceso de compilación

Fases de compilación: del código fuente al ejecutable



Resumen de las fases

Análisis léxico: convierte el texto fuente en una secuencia de *tokens* (unidades léxicas): identificadores, números, operadores, palabras clave. Es como separar una oración en palabras.

Análisis sintáctico: verifica que los tokens forman una estructura gramaticalmente válida y construye un *árbol de sintaxis abstracta* (AST). Es como verificar que las palabras forman una oración con sujeto, verbo y predicado.

Análisis semántico: verifica el *significado*: que las variables estén declaradas, que las operaciones sean válidas entre los tipos involucrados, que el alcance sea correcto.

Generación de código: traduce el AST a instrucciones del procesador destino (código máquina) o a un formato intermedio (bytecode).

10.2. Python y el bytecode: verificación con `dis`

Como vimos en la Semana 4, CPython compila nuestro código a bytecode y luego lo interpreta. Podemos observar el bytecode de una función de ordenamiento:

```
import dis
```

```
def swap_ejemplo(datos, i, j):
    '''Intercambia datos[i] y datos[j].'''
    datos[i], datos[j] = datos[j], datos[i]

dis.dis(swap_ejemplo)
```

4	0 LOAD_FAST	0 (datos)
	2 LOAD_FAST	2 (j)
	4 BINARY_SUBSCR	
	6 LOAD_FAST	0 (datos)
	8 LOAD_FAST	1 (i)
10	BINARY_SUBSCR	
12	ROT_TWO	
14	LOAD_FAST	0 (datos)
16	LOAD_FAST	1 (i)
18	STORE_SUBSCR	
20	LOAD_FAST	0 (datos)
22	LOAD_FAST	2 (j)
24	STORE_SUBSCR	
26	RETURN_CONST	0 (None)

Lectura del bytecode del swap

El swap de Python `datos[i], datos[j] = datos[j], datos[i]` se traduce en múltiples instrucciones de bytecode: primero carga ambos valores del lado derecho (`LOAD_FAST + BINARY_SUBSCR`), los rota en la pila con `ROT_TWO`, y luego los almacena en las posiciones inversas (`STORE_SUBSCR`). La PVM ejecuta cada instrucción secuencialmente —a diferencia de un procesador moderno que puede ejecutar múltiples instrucciones en paralelo (pipeline).

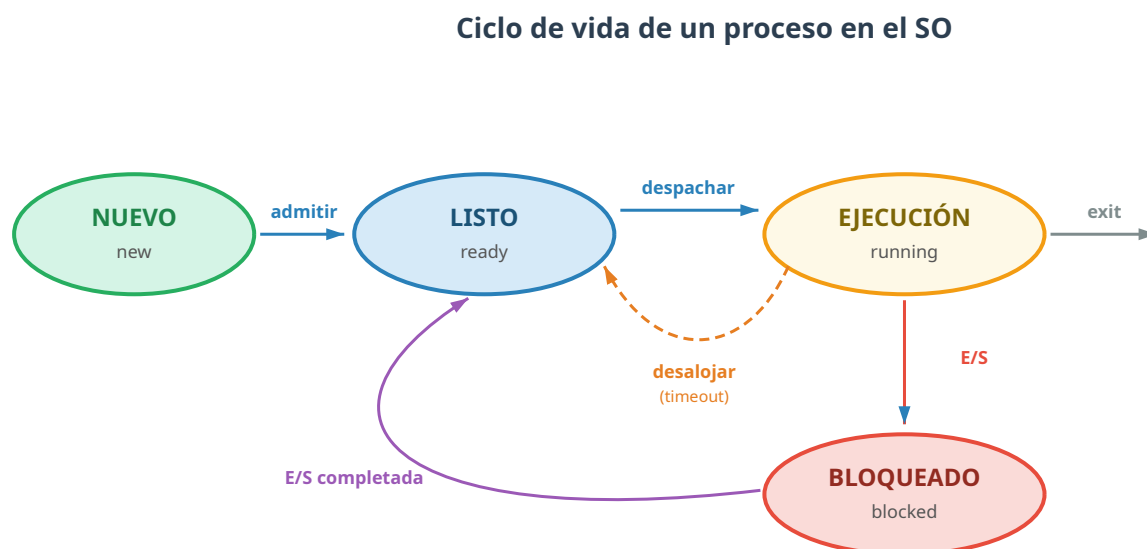
11. [Holístico] Procesos del SO y su Ciclo de Vida

Cuando ejecutamos nuestro programa de ordenamiento, el sistema operativo crea un **proceso** para gestionarlo. Comprender el ciclo de vida de un proceso

nos permite entender por qué un programa «se cuelga», qué sucede cuando leemos un archivo (el proceso se bloquea esperando el disco), y cómo el SO multiplexa la CPU entre múltiples programas.

11.1. Estados de un proceso

Un proceso transita por cinco estados fundamentales durante su vida:



11.2. Conexión con archivos y ordenamiento

¿Qué pasa cuando nuestro programa lee un archivo?

Cada operación de lectura —ya sea `archivo.read()`, `archivo.readline()` o la iteración con `for linea in archivo`— provoca que el proceso pase de **EJECUCIÓN** a **BLOQUEADO**: la CPU no puede continuar hasta que el disco (o el SSD) entregue los datos solicitados. El SO aprovecha este tiempo para **despachar otro proceso** que esté en estado LISTO. Cuando los datos están disponibles, nuestro proceso vuelve a LISTO y eventualmente retoma la ejecución. Este ciclo es *invisible* para el programador, pero explica por qué los programas que acceden mucho al disco son más lentos: no es que la CPU sea lenta, es que el proceso pasa mucho tiempo bloqueado esperando E/S.

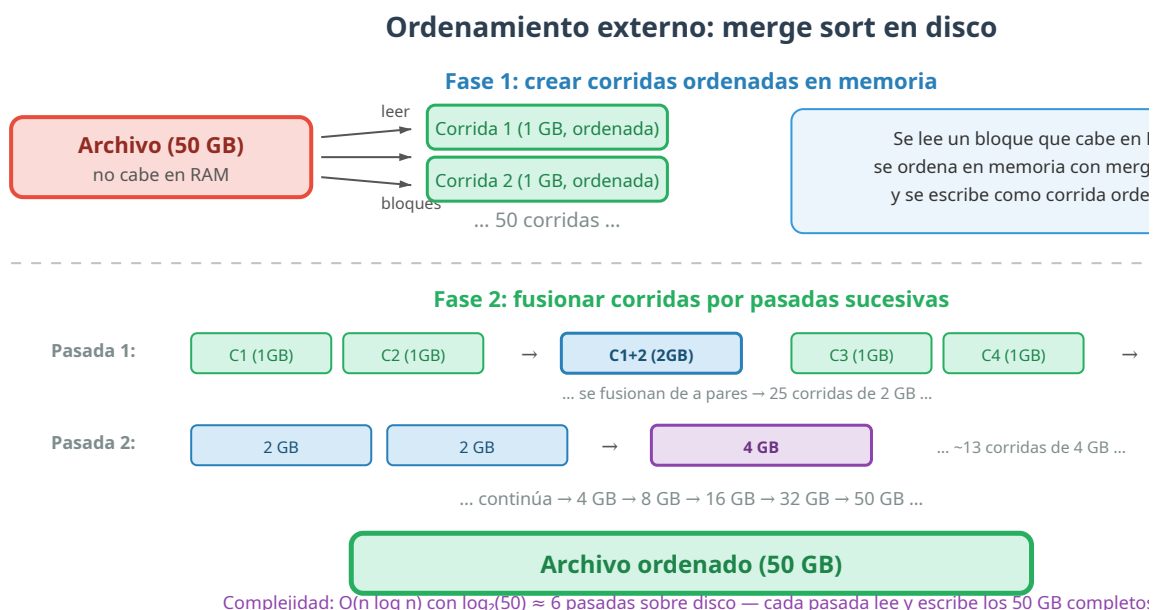
Impacto en el rendimiento del ordenamiento: ¿todo cabe en RAM?

Si el archivo cabe en la memoria RAM disponible, la estrategia óptima es leerlo completo a memoria (una transición EJECUCIÓN → BLOQUEADO → LISTO), ordenar en RAM sin más bloqueos de E/S, y escribir el resultado (otra transición). Esto es lo que hacemos con `archivo.read()` o `archivo.readlines()`, y es viable para archivos de tamaño moderado (hasta cientos de MB en máquinas actuales). Minimizar las transiciones a BLOQUEADO es un principio general de diseño eficiente.

Pero ¿qué ocurre si el archivo **no cabe en RAM**? Un registro de transacciones de 50 GB no puede cargarse en una máquina con 16 GB de memoria. Aquí la iteración línea a línea (`for linea in archivo`), que recomendamos en la sección 9.4, resuelve la *lectura* —pero no el *ordenamiento*: los algoritmos que estudiamos (selección, inserción, burbuja, merge sort) asumen que todos los datos están en RAM simultáneamente.

11.3. Ordenamiento externo (*external sorting*)

El **ordenamiento externo** es la familia de algoritmos diseñados para ordenar datos que **no caben en memoria principal**. El paradigma clásico es una adaptación directa de merge sort, lo que muestra la versatilidad del enfoque *divide y vencerás*.



Merge sort externo: por qué merge sort y no otro

El patrón de acceso de merge sort es **secuencial**: tanto en la lectura de las corridas como en la fusión, se recorren los datos de principio a fin. Esto se alinea perfectamente con las características del disco, que es rápido para lecturas secuenciales pero extremadamente lento para accesos aleatorios (*seek*). Los algoritmos cuadráticos (selección, burbuja) requerirían accesos aleatorios constantes al disco, y quick sort necesita particionar sobre posiciones arbitrarias. **Merge sort es el algoritmo de elección para ordenamiento externo** porque su naturaleza secuencial minimiza los costosos *seeks* de disco.

Impacto en el ciclo de vida del proceso

Durante el ordenamiento externo, el proceso alterna continuamente entre EJECUCIÓN (comparar y decidir qué elemento copiar) y BLOQUEADO (esperar datos del disco o esperar que la escritura se complete). En cada pasada de fusión se leen y se escriben *todos* los datos, generando una enorme cantidad de E/S. Por eso, minimizar el número de pasadas (fusionando más de dos corridas a la vez — fusión *k-way*) es una optimización crucial en la práctica.

12. Metodología Pólya: Ordenamiento como Ejemplo

Apliquemos las cuatro fases de Pólya al problema completo: «Dado un archivo `notas.txt` con un número de legajo y una nota separados por coma en cada línea, producir un archivo `notas_ordenadas.txt` con los datos ordenados por nota de mayor a menor».

Fase 1: Análisis

Comprender el problema y definir casos de prueba

Entrada: archivo `notas.txt` con formato `legajo,nota` por línea (ej: `12345,8`).

Salida: archivo `notas_ordenadas.txt` con las mismas líneas ordenadas por nota descendente.

Casos de análisis:

- **Normal:** varias líneas con notas variadas $\Rightarrow$ se ordenan de mayor a menor.
- **Límite:** archivo con una sola línea $\Rightarrow$ el resultado es idéntico a la entrada.
- **Límite:** varias líneas con la misma nota $\Rightarrow$ el orden relativo puede preservarse si el algoritmo es estable.
- **Extremo:** archivo vacío $\Rightarrow$ el resultado es un archivo vacío.

Fase 2: Diseño

Descomposición modular y pseudocódigo

Módulos:

```
leer_notas(ruta)  $\rightarrow$  lista de tuplas (legajo, nota)
merge_sort_por_nota(datos)  $\rightarrow$  lista ordenada por nota
descendente
escribir_notas(ruta, datos)  $\rightarrow$  escribe al archivo
```

Programa principal:

```
Prólogo: registros  $\leftarrow$  leer_notas('notas.txt')
Resolución: registros_ord  $\leftarrow$ 
merge_sort_por_nota(registros)
Epílogo: escribir_notas('notas_ordenadas.txt',
registros_ord)
```

Fase 3: Codificación

Implementar el diseño en Python

La implementación sigue exactamente la descomposición del diseño. Cada función tiene su docstring con precondiciones y poscondiciones. Se usa merge sort porque necesitamos estabilidad (alumnos con igual nota mantienen el orden original del archivo).

Fase 4: Evaluación

Verificar con los casos de análisis

Ejecutar el programa con cada caso de análisis definido en la Fase 1 y comparar la salida esperada con la obtenida. Verificar que el archivo de salida tiene el mismo número de líneas que el de entrada (no se pierden registros) y que las notas están en orden no creciente.

13. Visión de Conjunto

Esta semana hemos completado el binomio fundamental de la algoritmia: **búsqueda** (Semana 5) y **ordenamiento** (Semana 6). Estudiamos tres algoritmos cuadráticos —selección, inserción y burbuja— que, a pesar de su ineficiencia para grandes volúmenes, son conceptualmente reveladores: cada uno ilustra una estrategia diferente (seleccionar, insertar, intercambiar) y exhibe propiedades distintas (adaptabilidad, estabilidad, cantidad de swaps). Merge sort nos mostró que la barrera $O(n^2)$ no es inevitable: la estrategia *divide y vencerás* logra $O(n \log n)$ al precio de memoria auxiliar $O(n)$. Quick sort, con la misma complejidad promedio, ofreció una perspectiva complementaria donde el trabajo se concentra en la partición y no en la fusión, pero también nos mostró que una mala elección de pivote puede degradar el rendimiento a $O(n^2)$.

Los archivos de texto nos abrieron la puerta a la **persistencia de datos**: leer, procesar y escribir información que sobrevive al programa. El context manager `with` garantiza un manejo seguro de recursos, y la iteración línea a línea permite

procesar archivos de cualquier tamaño con memoria constante. Cuando los datos no caben en RAM, el **ordenamiento externo** —basado en merge sort— muestra que la elección del algoritmo depende no solo de la complejidad teórica sino también de la interacción con la jerarquía de memoria.

En el plano holístico, las fases de compilación nos mostraron el camino completo del código fuente hasta la CPU, y el ciclo de vida de procesos nos explicó por qué las operaciones de E/S (como leer un archivo) hacen que nuestro programa «espere» —y cómo el SO aprovecha esa espera para ejecutar otros procesos.

Conexiones del tema



Semanas 1–5: La notación O grande (Semana 5), la modularización y abstracción funcional (Semana 4), los patrones de procesamiento de secuencias (Semana 3) y la comprensión de la ALU y la jerarquía de memoria son requisitos directos para entender el diseño, implementación y análisis de los algoritmos de ordenamiento.



Semana 6 (este tema): Algoritmos de ordenamiento comparativos (selección, inserción, burbuja — $O(n^2)$), merge sort y quick sort (divide y vencerás — $O(n \log n)$), análisis de pivote y caso degenerado. Archivos de texto: open, read, write, with. Ordenamiento externo. Fases de compilación. Ciclo de vida de procesos del SO.



Semana 7: Punteros, referencias y modelo de memoria. Veremos cómo Python gestiona los objetos en memoria (id(), mutabilidad, aliasing), lo que es crucial para entender por qué el swap funciona y qué sucede cuando pasamos listas como argumentos a funciones de ordenamiento.



Semanas posteriores: La **recursividad** (Semana 8) formalizará el mecanismo que subyace a merge sort y quick sort. Las **estructuras de datos** (Semana 9) nos darán listas enlazadas, pilas y diccionarios, donde el ordenamiento tiene implicaciones

diferentes. En la Semana 11 se profundizará en quick sort con análisis de pivote y backtracking. El **proyecto final** (Semanas 13–16) exigirá elegir y justificar el algoritmo de ordenamiento según las características del problema.

Conexión holística: ordenamiento, hardware y sistema operativo

El rendimiento de un algoritmo de ordenamiento no depende solo de su complejidad teórica. La **localidad de referencia** afecta el uso de la caché: inserción accede a posiciones contiguas (buena localidad espacial), mientras que merge sort crea y recorre listas auxiliares que pueden no estar contiguas en memoria (peor localidad). A nivel del **SO**, ordenar un archivo que no cabe en RAM requiere *ordenamiento externo*, donde los datos se leen y escriben en disco por bloques —y aquí merge sort es la base de los algoritmos de ordenamiento externo porque su patrón de acceso secuencial se adapta bien a las características del disco.

Las **fases de compilación** (análisis léxico, sintáctico, semántico, generación de código) son el puente entre el código fuente que escribimos y el bytecode que la PVM interpreta. Cada vez que ejecutamos `python ordenamiento.py`, CPython realiza estas fases internamente antes de comenzar la ejecución. El **ciclo de vida del proceso** nos muestra que nuestro programa no vive en aislamiento: comparte la CPU con otros procesos, y las operaciones de E/S (como leer un archivo) causan transiciones de estado que el SO gestiona transparentemente.